MODULE 2

Methodology

1. Understand Morphological Analysis

Objective

To analyze the internal structure of words and understand how words change form based on tense, number, and meaning.

Steps

1. **Input from Preprocessed Data**
   - Use the cleaned and tokenized text obtained from the preprocessing module.
   - Ensure the text contains only valid tokens.
2. **Identify Morphological Components**
   - Identify inflected words such as *processing, studying, learning*.
3. **Apply Stemming**
   - Apply Porter Stemmer to remove suffixes.
4. **Apply Lemmatization**
   - Apply lemmatization to obtain dictionary base forms.
5. **Compare Results**
   - Compare original, stem, and lemma forms.
6. **Result Representation**
   - Store results in tabular format.
7. **Analysis**
   - Morphological analysis reduces lexical variations and improves frequency-based analysis.

CODE:

```
stemmer = PorterStemmer()

lemmatizer = WordNetLemmatizer()

morph_results = []

for word in tokens[:30]:

    stem = stemmer.stem(word)

    lemma = lemmatizer.lemmatize(word)

    morph_results.append([word, stem, lemma])

morph_df = pd.DataFrame(morph_results, columns=["Word", "Stem", "Lemma"])
```

```
display(morph_df)
```

OUTPUT:

	Word	Stem	Lemma
0	subjective	subject	subjective
1	this	thi	this
2	yearold	yearold	yearold
7	complaint	complaint	complaint
8	of	of	of
9	allergies	allergi	allergy
19	seattle	seattl	seattle
20	but	but	but
21	she	she	she
22	thinks	think	think
23	they	they	they
24	are	are	are
25	worse	wors	worse
26	here	here	here
27	in	in	in
28	the	the	the
29	past	past	past

2. Explore Language Models – n-Gram Model

Objective

To statistically model language and accurately predict the next word using n-gram probabilities.

Corpus Used

natural language processing is important

natural language processing is useful

natural language processing helps students

Steps

1. Corpus Selection

- The above preprocessed corpus is used as input.

2. Token Sequence Preparation

- Tokens are arranged sequentially to preserve context.

3. Generate n-Grams

CODE:

```
unigrams = Counter(ngrams(tokens, 1))

bigrams = Counter(ngrams(tokens, 2))

trigrams = Counter(ngrams(tokens, 3))

print("Total Unigrams:", len(unigrams))

print("Total Bigrams:", len(bigrams))
```

```
print("Total Trigrams:", len(trigrams))
```

OUTPUT:

Total Unigrams: 36884

Total Bigrams: 299755

Total Trigrams: 631278

Total Fourgrams: 833258

Unigrams

```
unigram_df = pd.DataFrame(
    [(k[0], v) for k, v in unigrams.items()],
    columns=["Unigram", "Count"]
)
print("\n--- Sample Unigrams ---")
display(unigram_df.sort_values(by="Count", ascending=False).head(10))
```

OUTPUT:

	Unigram	Count
24	the	147666
29	and	82610
83	was	71765
8	of	59009
12	to	50156
123	a	42276
6	with	35728
16	in	32626
70	is	26377
620	patient	21920

Bigrams

CODE:

```
bigram_df = pd.DataFrame(
    [(k[0] + " " + k[1], v) for k, v in bigrams.items()],
    columns=["Bigram", "Count"]
)
print("\n--- Sample Bigrams ---")
display(bigram_df.sort_values(by="Count", ascending=False).head(10))
```

OUTPUT:

	Bigram	Count
1255	the patient	19434
1181	of the	19169
27	in the	12777
1251	to the	12410
1297	was then	6962
107	and the	6369
1256	patient was	6289
1091	the right	5401
1265	on the	5290
1114	with a	4852

Trigrams

```
trigram_df = pd.DataFrame(
    [(" ".join(k), v) for k, v in trigrams.items()],
    columns=["Trigram", "Count"]
)
print("\n--- Sample Trigrams ---")
display(trigram_df.sort_values(by="Count", ascending=False).head(10))
```

OUTPUT:

	Trigram	Count
1439	the patient was	5804
2048	the patient is	3007
1219	as well as	2243
1393	is a yearold	1803
1329	there is no	1648
1444	the operating room	1506
1486	prepped and draped	1483
2674	was used to	1480
2049	patient is a	1480
1487	and draped in	1372

4. Probability Estimation

Bigram Conditional Probability Formula

$$P(w_2 \mid w_1) = \frac{\text{Frequency}(w_1, w_2)}{\text{Frequency}(w_1)}$$

CODE:

```
def bigram_probability(w1, w2):
        if (w1, w2) in bigrams and (w1,) in unigrams:
                return bigrams[(w1, w2)] / unigrams[(w1,)]
        return 0
```

Probability Table

All predictions for 'patient':

Index	Next Word	Probability
0	was	0.286907
1	is	0.151688
2	has	0.064553
3	tolerated	0.045347
4	had	0.040511
...	...	...
571	routinely	0.000046
572	reason	0.000046
573	landed	0.000046
574	title	0.000046
575	admitting	0.000046

6. Next Word Prediction

Input

Patient

Prediction

	Next Word	Probability
0	was	0.286907
1	is	0.151688
2	has	0.064553
3	tolerated	0.045347
4	had	0.040511
...	...	...
571	routinely	0.000046
572	reason	0.000046
573	landed	0.000046
574	title	0.000046
575	admitting	0.000046

Justification

$$P(was \mid patient) = 0.286907 \text{(highest probability)}$$

7. Model Comparison

- Unigrams capture word importance only.
- Bigrams improve prediction accuracy.
- Trigrams provide better context but require larger data.

8. Output Representation

- Frequent n-grams and predicted words are displayed.

3. Implement POS Tagging

Objective

To assign grammatical categories to words for identifying content-rich terms in text summarization.

Steps

1. Sentence selection from preprocessed corpus
2. Tokenization
3. POS tagging
4. Tag interpretation
5. Result visualization
6. Evaluation
7. Application analysis

CODE:

```
pos_tags = nltk.pos_tag(tokens[:500])

pos_df = pd.DataFrame(pos_tags, columns=["Word", "Tag"])

medical_terms = pos_df[pos_df['Tag'].str.startswith(('NN', 'JJ'))]

display(medical_terms.head(15))

print("\nPOS Distribution in Sample:")

print(pos_df['Tag'].value_counts().head(10))
```

OUTPUT:

	Word	Tag
0	subjective	NN
2	yearold	JJ
3	white	JJ
4	female	NN
5	presents	NNS
7	complaint	NN
9	allergies	NNS
14	allergies	NNS
19	seattle	NN
25	worse	JJR
29	past	JJ
33	claritin	NN
35	zyrtec	NN
39	short	JJ
40	time	NN

EXPANSION:

NN – Noun, singular or mass

JJ – Adjective

NNS – Noun, plural

IN – Preposition or subordinating conjunction

PRP – Personal pronoun

VBZ – Verb, 3rd person singular present

RB – Adverb

CC – Coordinating conjunction

DT – Determiner

VBD – Verb, past tense

POS Distribution:

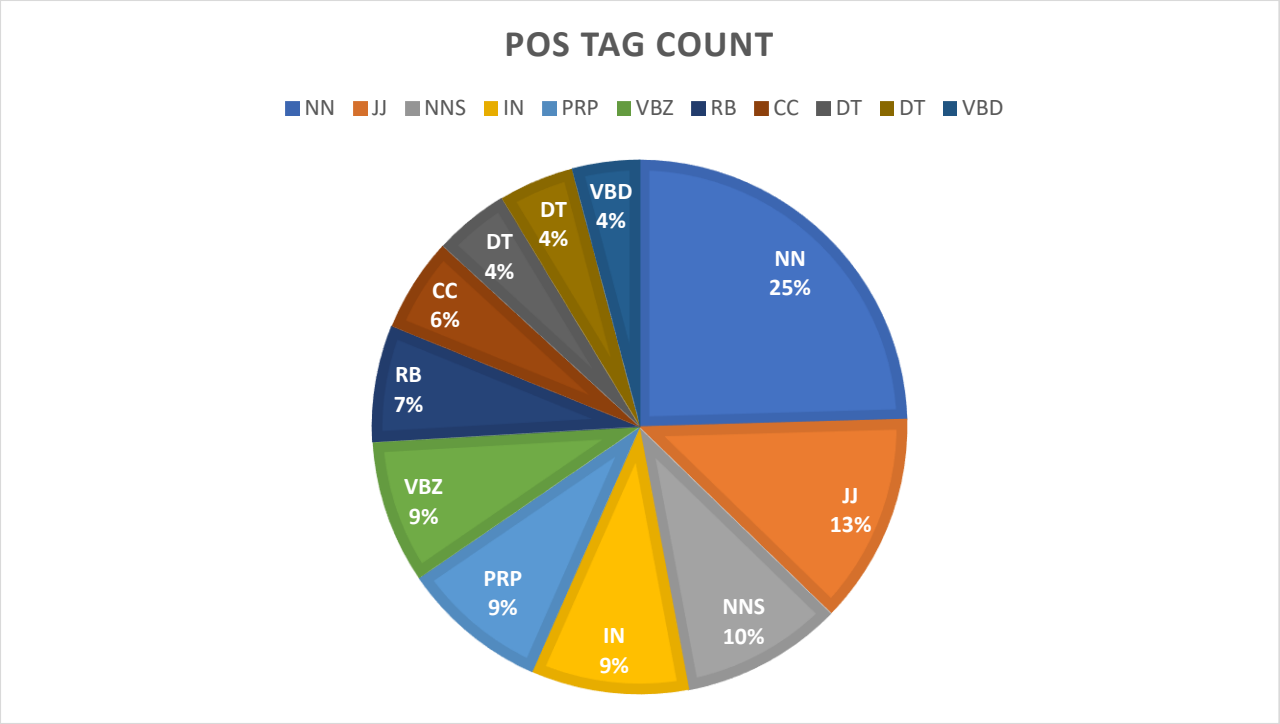

Fig 2.1: Pie Chart for POS Tag Count

RESULT:

The module was successfully implemented by performing morphological analysis, n-gram modeling with next-word prediction, and POS tagging for linguistic analysis.